WorkPulse 2.0

Employee Daily Work Updates, Attendance, Task Management & Performance Evaluation | Updated project documentation | October 2026

1. Technology Stack

Layer	Technology / Purpose
Frontend	React, TypeScript, Vite, HTML, CSS, Axios
Backend	PHP 8.4 in Docker; Laravel 13; REST-style JSON API
Database	MariaDB 11.4 in Docker (MySQL-compatible)
Authentication	Laravel session-based authentication and Sanctum CSRF-cookie workflow
Deployment / Dev	Docker Compose, Nginx, PHP-FPM; native XAMPP option
Development tools	Windows PowerShell, VS Code, Composer, npm, Git / GitHub

2. Demo Login Credentials

Local demonstration only. The credentials below were provided for testing. Change these passwords before publishing or sharing a deployed application.

Role	Email	Password
Administrator	admin@workpulse.local	ChangeMe@2026!
Manager	test1@gmail.com	123456654321
Employee	test@gmail.com	123456654321

Credential note: The earlier employee credential was typed in an ambiguous combined format. This document interprets it as test@gmail.com with password 123456654321; verify against the actual database if login fails.

3. Application Overview

WorkPulse 2.0 is a role-based workforce management application. Administrators manage employees and tasks, managers monitor their assigned teams, and employees record attendance and review work assigned to them. The system includes performance monitoring, reporting, and settings screens.

4. Access URLs (Docker Local Environment)

Purpose	URL
Frontend - preferred local domain	http://workpulse.local:5174/login
Backend API - preferred local domain	http://api.workpulse.local:8080
Frontend - localhost alternative	http://localhost:5174
Backend - localhost alternative	http://localhost:8080

The custom local domain requires Windows hosts-file entries mapping workpulse.local and api.workpulse.local to 127.0.0.1. These addresses are for development on the configured computer, not public internet hosting.

5. Roles and Main Capabilities

• Administrator: Dashboard, employee management, task assignment, attendance monitoring, performance analytics, reports and application settings.

• Manager: Team overview, assigned team members, team tasks, attendance summaries and performance reviews; task access is limited by backend authorization rules.

• Employee: Personal workspace, daily check-in/check-out, attendance history and personally assigned tasks (where the task panel is visible and functional).

6. Task Assignment and Status Flow

An administrator can create tasks and assign them to active employees or managers. The task record stores the assignee, creator, title, priority, dates, status and progress. Managers may create or manage tasks for authorized team members. The backend filters visible tasks by role.

Supported statuses: todo, in_progress, in_review, completed, blocked. Priorities: low, medium, high, urgent. Completing a task requires progress = 100%; progress = 100% is not accepted for non-completed tasks.

Personal task endpoint: GET /api/my-tasks returns tasks assigned to the authenticated user. The React MyTasks component calls this endpoint and can update progress through PATCH /api/tasks/{id}/status. The previous session did not conclusively verify the task panel placement in both employee and manager interfaces.

7. Attendance and Performance

Employees can check in and check out through the employee workspace, with a dated attendance history. The manager dashboard aggregates team-member count, task counts, presence/late/not-checked-in metrics, and finalized performance reviews. The administrator performance screen displays employee and departmental metrics and task-completion summaries.

8. High-Level Architecture

Browser: React + Vite at workpulse.local:5174 → HTTP requests: Axios with credentials / XSRF support → API: Nginx at api.workpulse.local:8080 → Application: Laravel 13 / PHP-FPM → Storage: MariaDB database workpulse_docker.

Laravel authenticates users, checks their roles, validates inputs, and queries database records. The frontend renders the appropriate admin, manager or employee interface.

9. Database Modules

The Docker database includes users, employees, tasks, attendances, performance_metrics, performance_reviews, app_settings, sessions, personal_access_tokens, and Laravel support tables for migrations, cache, queues and password reset tokens. The prior verified environment contained 16 tables and completed migrations.

10. Docker Setup and Run (Windows PowerShell)

From the repository root E:\Projects\WorkPulse-2.0, with Docker Desktop running:

```
docker compose --env-file .env.docker up -d --build
docker compose --env-file .env.docker ps
docker compose --env-file .env.docker exec backend php artisan migrate:status
docker compose --env-file .env.docker exec frontend npm run build
```

Open http://workpulse.local:5174/login. For a new installation, configure environment values, install dependencies and run migrations before logging in. Do not use migrate:fresh or docker compose down -v on a database containing data.

11. Environment Configuration

The Docker environment uses frontend VITE_API_URL=http://api.workpulse.local:8080, backend APP_URL=http://api.workpulse.local:8080, FRONTEND_URL=http://workpulse.local:5174, SANCTUM_STATEFUL_DOMAINS including both local hosts and ports, SESSION_DOMAIN=.workpulse.local, and SESSION_SECURE_COOKIE=false for local HTTP development.

CORS must permit the frontend origin and credentials; Axios is configured with withCredentials and withXSRFToken. Before POST /api/login, the frontend obtains a CSRF cookie from /sanctum/csrf-cookie. The earlier HTTP 419 issue was resolved after aligning domains and cookie/session configuration.

12. API Endpoints (Verified from Project Discussion)

Method / Route	Purpose
POST /api/login	Sign in (CSRF/session protected)
GET /api/me	Current authenticated user
GET /api/my-tasks	Tasks assigned to current user
PATCH /api/tasks/{id}/status	Update task status and progress
GET /api/manager/dashboard	Manager team overview
GET /api/tasks	Role-filtered task list
POST /api/tasks	Create a task (admin/manager)

Other endpoints exist for employees, attendance, reviews, reports and settings; consult backend/routes/api.php for the authoritative route inventory.

13. GitHub Publishing and Security Checklist

• Commit application source, Dockerfiles, Compose configuration, migrations, documentation and a sanitized environment example.

• Never commit: .env, .env.docker with real secrets, APP_KEY, database passwords, private credentials, node_modules, vendor, or database dumps containing user information.

• Rotate demonstration passwords and secrets before any public deployment. This PDF itself contains demo passwords; do not publish it in a public repository without removing that section.

• Run the frontend build and Laravel checks, then test login, role permissions, task creation, attendance and reporting.

14. Current Project Status and Next Steps

Confirmed working: Docker services, local-domain frontend access, admin and manager dashboard data, employee attendance UI, database migrations and login after the domain/CSRF fix. Further verification recommended: visibility and usability of the My Assigned Tasks panel for employee and manager accounts, end-to-end task updates, production-ready configuration, and GitHub publishing.